CHOREO

算子专用编程语言介绍系列

编程篇之贰：结合并行与迭代

迭代与多级别的并行

大块数据在算子实现中往往需要通过数据拆分（Tiling/Blocking）运算的方式来进行处理。而要高性能处理每个拆分后的数据分块，就需要使用并行的硬件资源。并行的硬件资源，无论是任务级并行的线程还是指令集并行的向量指令，其能够达到的并行处理数量始终是有上限的。这通常导致的数据分块的处理需要**同时考虑并行与迭代**。

在 Choreo 中，同时考虑数据分块的并行与迭代并不复杂，我们可以看看下面的示例代码：

```
1  __co__ auto ele_add(s32 [M] lhs, s32 [M] rhs) {
2    s32 [M] output;
3    parallel p by 2 {       // grid级别上并行度为2
4      foreach x in [16] {   // 进行16次迭代
5        lfs = dma.copy.async lhs.chunkat(x # p) => shared;
6        rfs = dma.copy.async rhs.chunkat(x # p) => shared;
7        wait lfs, rfs;
8
9        parallel q by 12 { // block级别上并行度为12
10         lfl = dma.copy.async lfs.chunkat(q) => local;
11         rfl = dma.copy.async rfs.chunkat(q) => local;
12         wait lfl, rfl;
13
14         local s32 [lfl.span] l1_out;
15         foreach i in [l1_out.span] // 迭代处理l1_out的所有元素
16           l1_out.at(i) = lfl.data.at(i) + rfl.data.at(i);
17
18         dma.copy l1_out => output.chunkat(x # p # q);
19       }
20     }
21   }
22   return output;
23 }
```

可以看到，这个名为 `ele_add` 的 *Tileflow* 的函数中，我们使用`.chunkat(x # p)`这样的表达式进行大块数据拆分。在 Choreo 中，我们称 `x` 与 `p` 为**有界变量**（bounded variable），因为其我们知道它们的上界（分别为 2 与 16）。而 `x` 也是循环迭代的**循环变量**，而 `p` 则被称为**并行变量**，其唯一地标识了一个并行实体。这里的表达式 $x \# p$ 则结合了并行与迭代，对数据的单一维度进行切分。进而实现了并行与迭代在数据切分，以及数据下标计算的同时实施。我们可以参考下页的图进行理解。

除此之外，我们在代码中看到了 **多级别的并行**——这对于 GCU 的编程者并不陌生。如果使用 CUDA 的术语，就是 grid 和 block 两个级别。在这段代码中，我们在 grid 级别并行地（使用两个执行线程）将数据分块搬入 block 共享的（*shared*）内存，而在 block 级别，每个 block 中的 12 个执行线程（由 `p` 标识，上界为 12）则分别将数据搬入线程独占的（*local*）内存，从而完成多级别并行的数据搬运以及相关运算。

Tileflow 代码逐行解析

- **函数定义**：代码行 1 定义了名为 `ele_add` 的 Choreo *Tileflow* 函数，它接受两个**动态形状**的一维张量 `lhs` 和 `rhs` 作为输入，返回值则需要编译器推导得出。
- **数据读入共享存储**：代码行 3–4 设定了程序第一级别的并行度为 2，循环迭代次数为 16。而对于每个循环迭代。代码行 5 – 7 则通过异步的数据拷贝操作（`dma.copy.async`），使得每个线程组在当前循环迭代中，获得一块大小为 $M\ /\ (\#x\ *\ \#q)$，且不与其它线程组及迭代重合的数据块 ⋆。故而数据块的大小实际为$M\ /\ (2\ *\ 16)$。
- **数据读入私有存储**：代码行 9 声明了第二级别的并行度为 12，即每个线程组中使用 12 个线程进一步的并行。类似地，我们在代码行 10 – 12 中使用异步数据拷贝，使得每个线程搬运共享数据中互不重叠的部分数据到线程的私有存储中。
- **数据运算与写回** 代码行 14 声明了一块临时存储 `l1_out`，用于存储代码行 15 – 16 中计算的数据的逐元素之和。之后代码行 18 的 `dma.copy` 语句将计算结果从本地私有存储直接同步地写回到全局数据中。注意这里使用了表达式$x \# p \# q$来定位分块在全局存储中的位置。

⋆ 这里$\#x$, $\#q$中的单元操作符$\#$是**取上界**操作，本例中即为 12 与 2。

CHOREO

动静皆宜的断言（assert）

Choreo 编译器能够依据语法规则产生产生大量的断言（assert）以保障程序的安全性。而有的时候，程序员知道更多的上下文，那么也可以主动地进行防御式编程。比如：

```
__co__ void foo(f32 [M, N]) {
  assert(M > 128,
         "unexpected M.");
}
```

程序员通过主动的断言，以使得调用 foo 且参数二维张量数据的高维度大于 128 的情况及时退出。这么一来，程序员可以及时地发现程序调用了不合适版本的算子。

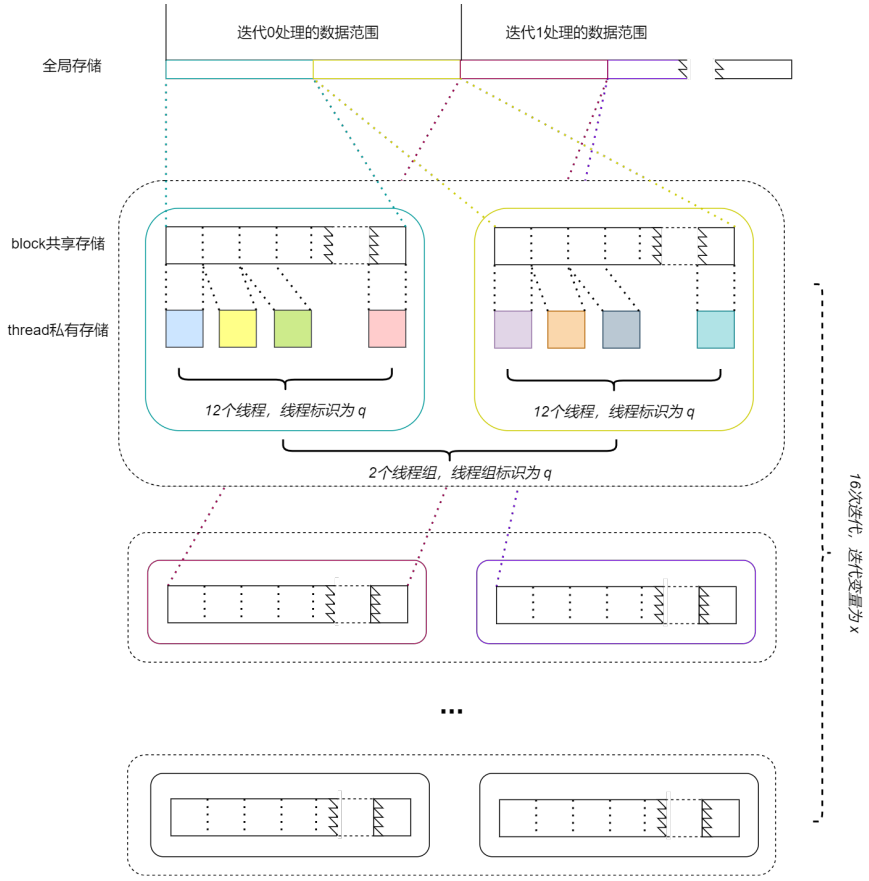
而有的时候，代码维护者在修改代码时可能并不完全了解代码作者的企图，比如：

```
#define M 32
__co__ void foo(f32 [M, N]) {
  assert(M > 128,
         "unexpected M.");
}
```

这里，程序员定义了一个宏，使得代码中 M 维度变为 foo 函数不可接受的静态维度，这时候 Choreo 编译器会在**编译时期就会触发断言**，中断编译并汇报错误信息，以帮助程序员更早地发现程序中地错误。

欢迎加入 Choreo 服务群，汇报 Choreo 的问题，分享试用体验，以及提出建议

图示：多级线程与迭代



如图所示，在示例代码中，算子通过迭代的方式对数据做了最大颗粒度的划分。这里，总共 16 次的迭代将一维的数据等分为 16 份。而每次迭代处理的数据则进一步被两个线程组等分。每个线程组使用一个线程发起 DMA 拷贝操作，将相应的数据到线程组共享内存。而线程组中的 12 个线程则将共享的数据进一步等分为 12 份，并拷贝相关到线程的私有存储进行进一步的并行处理。

在 Choreo 中，总是可以通过类似于 $x \# p$ 的表达式 *，**对单一的数据维度进行多级的切分**。更为具体地，在 foreach 与 parallel-by 等结构中，形如：

```
parallel p by 2
  foreach x in [16]
    dma.copy a.chunkat(x # p) => shared;
```

这样的代码，其等效于以下的循环/并行结构：

```
for (x = 0; x < 16; x++)
 for (p = 0; p < 2; q++) {     // 对该层次进行并行化
  size_t chunk_size = a.size() / (16 * 2);
  __shared__ u8 buffer[chunk_size]; // 伪码，编译器不支持动态数组
  size_t offset = (x * 2 + p) * chunk_size;
  dma_copy(dst, &(a + offset), chunk_size);
 }
```

可以看到，Choreo 编写的代码简单了很多：数组下标运算，动态形状的存储分配等等琐碎、例行性甚至难于实现的代码都可以由 Choreo 编译器隐式地生成。这让代码开发易于上手，也让代码维护简单——不必战战兢兢地处理每个细节了。

使用 Choreo 编写算子，**程序员可以从很多底层细节的实现中摆脱出来，更多地关注数据切分等与高层抽象相关的算法设计**。这也是 Choreo 设计的主要目的之一。

---

* 注意：$x \# p$与$p \# x$是不同的操作。在 *chunkat* 表达式中，前者可以认为是将数据先切分为 x 份，再切分为 p 份，进而其下标访问语义为$x * \#q + q$，而后者的下标访问为$q * \#x + x$。

深入了解 Choreo

如果想深入了解 Choreo，我们准备了相关的文档、视频教程供您学习：

- 网页教程：http://choreo.enflame.cn/
- 视频教程：https://learning.enflame-tech.com/course/144

参与到 Choreo 的开发中

无论是汇报问题，或是建议新的 Choreo 特性，都可以通过在 Choreo 项目主页创建 issue 的方式达成：http://git.enflame.cn/xiaofeng.guan/choreo/-/issues

Choreo 的**项目主页**在：http://git.enflame.cn/xiaofeng.guan/choreo, 其包含了编译使用 Choreo 的说明

如有需要请联系：

- 官孝峰：xiaofeng.guan@enflame-tech.com
- 石恒：heng.shi@enflame-tech.com

CHOREO

- 简化数据搬运编程
- 提高程序安全性
- 支持动态形状